Microbiology LIMS POC — Process at a Glance

v3.2 (Conditioned Discovery) | Reference to `microbio_lims_architecture.drawio` | 2026-04-29

What the system does

A subject-matter expert (SME) drops microbiology SOPs into S3. An agent pipeline reads them and produces two things for the dev team: **(A)** Jira-ready *Epics* and *Stories*, and **(B)** a per-culture *configuration profile* in YAML (one each for urine, blood, target-pathogen).

The existing Cytology Jira backlog is used as a **teaching corpus** — example (SOP excerpt → Story) pairs the agent learns the house style from. It is *not* a registry to match against. The agent stops at Story. *Tasks* (test IDs, library versions, DB columns) are dev-authored — the agent has no codebase visibility to fabricate them defensibly.

The pipeline — 5 agents, run as a single batch over the SOP cohort

1. **Epic Extractor** reads the cohort and identifies capability domains, running in v3.2 *conditioned mode* (matches drafts against the prior epic catalog before clustering anything novel; see §v3.2 below). *Out:* `{id, title, discipline, themes[], theme_catalog_version, epic_analog?}`. Example title: *Microbiology Specimen Receipt*.
2. **Story Extractor** reads each SOP and emits Stories. Each Story declares one of **four shapes**, because real SOPs produce a mix — a single rubric over-fits:
   - **Capability** (a configurable feature; uses MUST/SHALL) — e.g. *Specimen Rejection Rules Engine*.
   - **Workflow-stage-split** (same capability, decomposed by stage; siblings cross-linked) — e.g. *PHI Update Before Work Has Started / After Work Has Started / After Results Reported*.
   - **Configuration-instance** (concrete values for one culture; this is data, not capability language) — e.g. *Urine centrifugation: speed 1500 g, duration 5 min, temp 4 °C*.
   - **Cleanup / correction** (modify or remove an existing artifact) — e.g. *Remove Result Bucket option from Add/Modify QT code screens*.
3. **Validator — gate #1** verifies the declared shape, then applies a *shape-specific* rubric (capability requires MUST/SHALL + testable AC; config-instance requires concrete typed values + verbatim citation; cleanup requires named target + before/after + regression risk; stage-split requires stage in title + sibling enumeration). Two revision rounds; on exhaustion, escalate to SME with the failed checks visible.
4. **Cross-SOP Synthesis** runs only after gate #1 has cleared the entire batch. It clusters concrete stories whose members come from ≥ 2 **distinct SOPs** (two stories from the same SOP do *not* qualify) and emits one capability-shaped story per cluster, with `cross_links[]` pointing back at the contributing concrete stories. **Concrete stories are kept** — the capability story is added as a sibling, not a replacement.
5. **Validator — gate #2** (same agent as gate #1) re-validates the synthesized stories against the capability rubric — they are a *new* artifact, not a derivative of validated input.
6. **Dependency Resolver** — final pass. Resolves story-to-story dependencies, stage-split sibling links, and capability-to-child links from synthesis. Outputs Jira-ready artifacts.

Story schema (what gets pushed to Jira)

```
{
  id, epic_id, title, description,
  shape ∈ {capability, workflow-stage-split, configuration-instance, cleanup},
  acceptance_criteria[], source_citations[], dependencies[], cross_links[],
  estimated_complexity ∈ {S, M, L}, source_chunks[]
}
```

The substrate — how the agents see source material

Ingestion (offline, fires when an SOP lands in S3): *parse* (PDF / OCR for scans / DOCX / transcripts / Jira CSV) → *chunk* (section-aware, ∼150 chunks per typical SOP) → *enrich* (clinical NER for organisms / drugs / specimens, code expansion via UMLS, PHI detection & redaction, theme tag from G1–G8) → *embed* (both meaning-based vectors and keyword tokens for BM25) → *store* (OpenSearch for the indexes, DynamoDB for chunk + doc metadata, S3 keeps the originals as the citation target).

Retrieval (real-time, per chat query): normalize → *intent decode* into `{action, themes, discipline, granularity, confidence}` → expand with synonyms → *multi-slot retrieve* (in-theme target / Cyto analogy / adjacent themes / Jira exemplars / regulatory — each slot runs keyword + meaning search and fuses results) → rerank → assemble a role-labeled prompt (PRIMARY / ANALOGY / ADJACENT / EXEMPLAR / REG) → LLM generation with citations

resolved to S3 doc + section + version.

Two thresholds gate the chat to prevent hallucination:

- $\tau = 0.7$ *intent confidence* — below this, ask a clarifying question instead of retrieving.
- $\theta = 0.5$ *retrieval relevance* — if every slot falls below, refuse and offer the user three options (upload SOPs, draft from analogy with caveat, clarify scope).

Worked example — one query end-to-end

Query: *"Generate epics for specimen rejection in microbiology."*

Intent decode: `{action: generate, themes: [G1], discipline: Micro, granularity: epic, conf: 0.91}` ⇒ proceed.

Retrieve → rerank: 48 candidate chunks across 5 slots, trimmed to 20 by the reranker; low-score gate passes.

Assemble: 6,840-token prompt with the chunks role-labeled and the output schema attached.

LLM output (structured JSON, abbreviated):

```
{ epic_id: epic_draft_3, title: "Specimen Rejection Workflow", theme: G1,
  citations: ["s3://.../SOP-MICRO-007 §AC", "s3://.../SOP-CYTO-021 §Procedure", "CLSI M22 §3.4"],
  stories: ["Validate transport temperature", "Flag specimens older than 4 h", ...] }
```

Telemetry written: query id, intent, slot scores, total chunks (20), tokens (6,840), latency (∼4.3 s).

What ships

- **Output A — Jira artifacts.** Epics (with optional `cyto_epic_analog` annotation), per-SOP concrete Stories in all four shapes, synthesized capability Stories cross-linked to their children, and the dependency edges between them.
- **Output B — Per-culture YAML profile.** One profile per culture in scope. Every value is typed (units / enums / nullable rules) and cites the SOP excerpt it came from. Inline references to the related Story's AC give traceability without duplicating values into the Story body.

Example YAML excerpt (`cultures/urine.yaml`):

```
centrifugation:
  speed: {value: 1500, units: g} # cite: SOP-MICRO-014 §3.2 L41-43
  duration: {value: 5, units: min} # cite: SOP-MICRO-014 §3.2 L44
  temp: {value: 4, units: degC} # cite: SOP-MICRO-014 §3.2 L45
related_story_ac: STORY-1042#AC-2
```

v3.2 — Conditioned Discovery (warm-start theme & epic bootstrap)

v3.1 generated epics from-scratch and treated themes G1–G8 as a closed Cyto-derived vocabulary. v3.2 adds a *warm-start* step at both levels.

(1) Theme Discovery agent (new, pre-flight). Runs once per discipline (or when the G0 alarm trips). Four passes: **Classify** chunks against the prior catalog at τ_{match} (0.65) → **Cluster** only the residuals (size $\geq \varepsilon_{novelty} = 3$) → SME **ratifies** novel candidates (inherited bypass) → **Re-tag** the corpus on catalog version bump. Emits `<discipline>_v<N>`.

(2) Epic Extractor in conditioned mode. For each draft epic: classify against the prior epic catalog (`cyto_epic_v1`); if score $\geq \tau_{match}$, link `epic_analog` and reuse the existing epic ID by construction; otherwise the residual is clustered across SOPs and SME ratifies novel ones. Emits `<discipline>_epic_v<N>`.

Result. Most themes and epics inherit their structure with explicit lineage; only genuinely-novel structure (with cluster evidence) goes to SME. The dev team reads the output and immediately sees what's reused vs. new.

Worked example — bootstrapping Histology from `cyto_v1`: 460/600 chunks classify into G1–G8 (residual = 140). Pass 2 surfaces 4 novel themes (H1 Tissue Processing, H2 Staining, H3 Block Archive, H4 Frozen Section); 22 chunks → G0 (3.7%, below alarm). Pass 3 SME confirms all 4; flags G7 as redundant in histo (folded into H1) — discarded. Pass 4 re-classifies the 12 G7-orphaned chunks (most land in H1). Final `histo_v1`: 11 active themes (7 inherited + 4 novel; G7 in ANALOGY map only). 7/8 of the prior themes survived.

ANALOGY map artifact (new). Explicit cross-catalog link table replacing v3.1's implicit shared-theme assumption. Each link typed: `identical` / `partial` / `discarded_in_target` / `novel_in_target`. Drives the ANALOGY retrieval slot at query time.

Micro POC migration. Snapshot Cyto's themes as `cyto_v1` and Cyto's epic backlog as `cyto_epic_v1`. Run conditioned Epic Extractor over the 3 in-scope SOPs to produce `micro_epic_v1` (most epics inherit; AST is the likely novel). Stories carry `theme_catalog_version: cyto_v1` (Micro reuses the Cyto theme catalog wholesale; only the epic catalog forks). Emit a `cyto_epic_v1` ↔ `micro_epic_v1` ANALOGY map alongside.

Cross-cutting (one line)

KMS encryption at rest & in transit; IAM least-privilege with VPC endpoints; DLQ for parse / OCR / embed failures; CloudWatch metrics + alarms; CloudTrail audit log on every API call. *Generalization (v3.2):* when Histology arrives, Theme Discovery warm-starts `histo_v1` from `cyto_v1` pre-flight; the main pipeline then runs unchanged in shape.